


Advanced RAG for a Clothing Shopping Assistant

# Improving RAG Quality

SESSION ROADMAP

# Agenda

| | | |
|---|---|---|
| M0 | **Baseline** | The scoreboard everything else reports to |
| M1 | **Document Construction** | What text represents a catalog record |
| M2 | **Query Understanding** | Expansion + decomposition |
| M3 | **Attribute Extraction to Filters** | Structured constraints out of free text |
| M4 | **Hybrid Search** | Sparse + dense fusion |
| M5 | **Reranking** | Cross-encoder re-scoring of top candidates |
| M6 | **Context Assembly & Grounded Generation** | Dedup, diversity, cite-and-refuse |

ALSO COVERED (THEORY ONLY, NO HANDS-ON BUILD)

Query Routing · Excluding Vector Search Outliers · Embedding Model Fine-Tuning · LLM Fine-Tuning

BEFORE WE START

# If your RAG pipeline isn't working, it's usually not the LLM.

The LLM will give you the right answer as long as you hand it the right context. Almost every failure in this session is a retrieval problem wearing a generation costume. Fix retrieval first.

# Why Retrieval-Augmented Generation

- LLMs are frozen at training time — they don't know your catalog, your prices, or what shipped yesterday.
- Fine-tuning teaches new skills, but it's slow and expensive to keep current with a changing catalog.
- RAG grounds a general-purpose model in your live data at inference time, without retraining it.

# How Vanilla RAG works?

*Two Phases*

**Ingestion (Pre-processing)**

1) Collect data from docs, forums, knowledge bases

2) Chunk text + add metadata

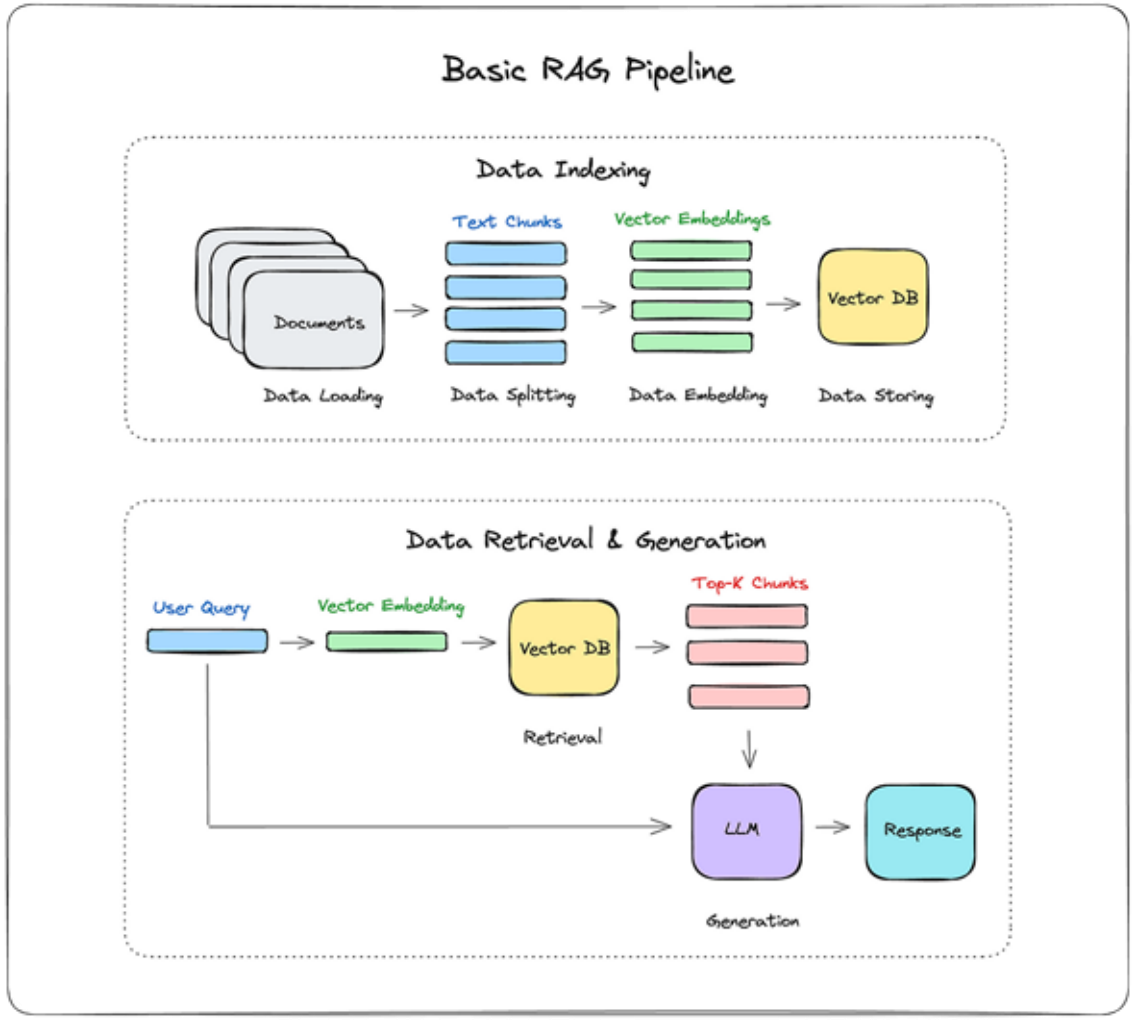
3) Embed chunks into vectors

4) Store in a vector DB

**Inference (Run-time)**

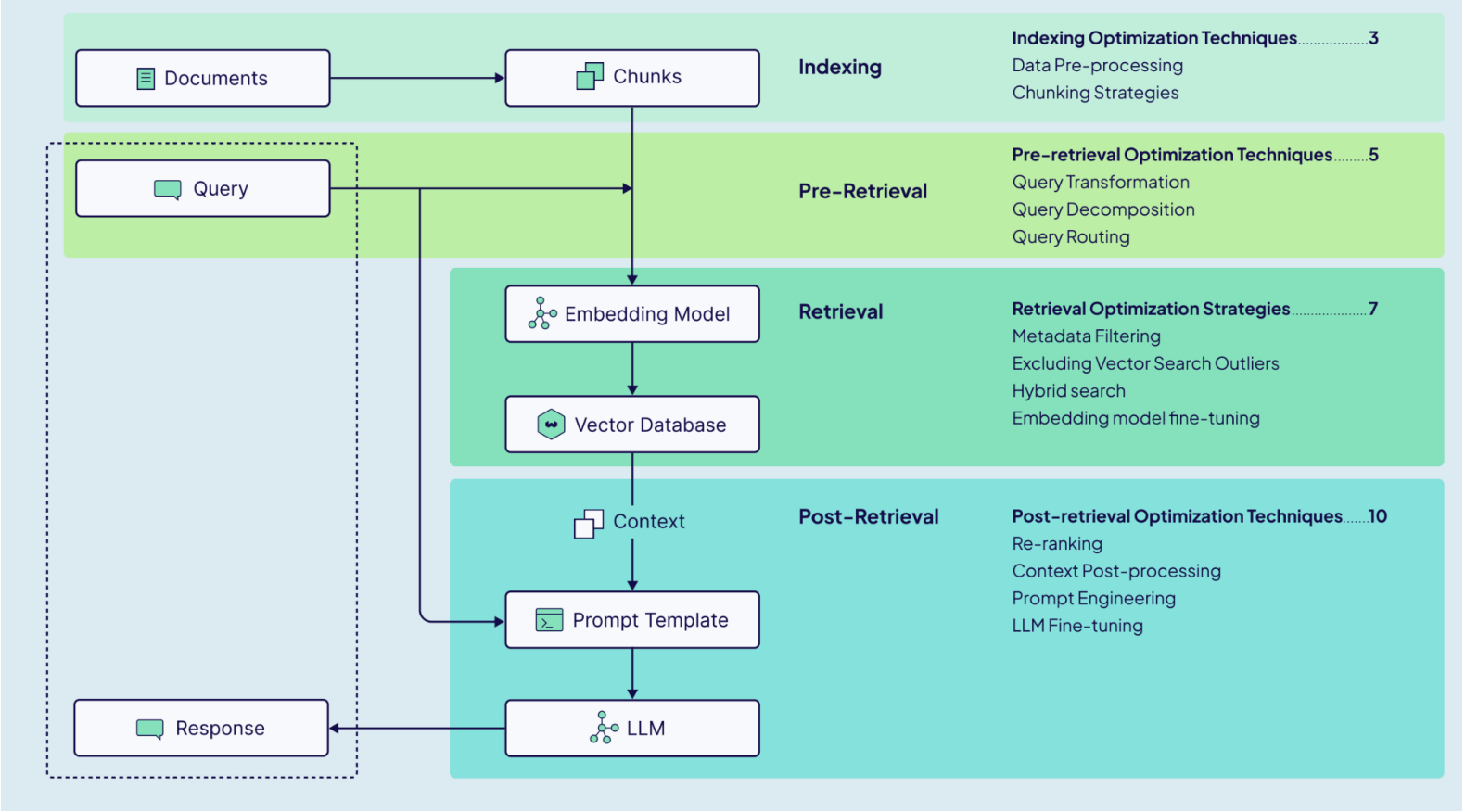
5) Retrieve similar chunks via cosine similarity

6) Add retrieved chunks to context and generate grounded answers with LLM



# The Improving RAG Pipeline

Every technique in this session sits in one of these four colored stages.




*Source: Weaviate*

# Where Naive RAG Breaks

- Indexing — the wrong text gets embedded, so nothing downstream can retrieve the right signal.
- Pre-Retrieval — the query as typed doesn't match how the catalog is written or scoped.
- Retrieval — pure similarity search can't enforce hard constraints or exact tokens.
- Post-Retrieval — even good hits arrive in the wrong order, duplicated, or ungrounded.

*This session is structured around exactly these four stages — one technique from each, measured.*

HANDS-ON MODULE 0

# Baseline & Evaluation Harness

→ Load the clothing catalog — 618 SKUs, 16 subcategories, single Apparel category.

→ Build a vanilla RAG pipeline: description-only embeddings, top-5, generate.

→ Run the demo queries and watch the baseline fail three different ways.

**DEMO QUERIES**

`"what should i wear to my friend's wedding,"`

**IDEAL** Retrieves suits and dress shoes (16 relevant SKUs) — not tees or socks. The baseline misses this entirely on vocabulary alone.

`"RN-4412"`

**IDEAL** The exact SKU with style code RN-4412, and only that one. Dense-only search returns near-miss codes RN-4142 / RN-4421 above it.

`"red running shoes under $50 in size 10"`

**IDEAL** SKUs that are actually red, size 10, and under $50 — not just 'running-shoe-flavored' results.

**PROBE: vocab_gap · exact_token · attribute_constrained**

# 01

# Indexing Optimization

Structuring data in more organized, searchable ways

# Data Pre-Processing: Extraction, Cleaning, Parsing

- Extraction & parsing — preserve structure while capturing the content that matters (text, scanned PDFs via OCR, HTML, metadata).
- Cleaning — strip boilerplate, fix inconsistencies, handle missing values.
- Transformation — convert everything to one standardized schema regardless of source format.
- For a product catalog this stage is mostly done for you — records already arrive structured.

# Why Chunking Matters — and Why It Doesn't Apply Here

- For long documents, chunk size trades off two things: retrieval precision (smaller = more targeted) against context preservation (larger = more coherent).
- That trade-off exists because a document is one long stream of text you have to cut somewhere.
- A catalog record isn't a stream — it's already a discrete unit. There's nothing to cut.
- The equivalent decision for structured data isn't 'where do I cut' — it's 'what do I embed.' That's Module 1.

# Choosing a Chunking Strategy

| Chunking Strategy | How It Works | Complexity | Best For | Examples |
|---|---|---|---|---|
| **Fixed-Size (or Token)** | Splits by token or character count. | Low | Small or simple docs, or when speed matters most | Meeting notes, short blog posts, emails, simple FAQs |
| **Recursive** | Splits text by repeatedly dividing it until it fits the desired chunk size, often preserving some structure. | Medium | Documents where some structure should be maintained but speed is still important | Research articles, product guides, short reports |
| **Document-Based** | Treats each document as a single chunk or splits only at document boundaries. | Low | Collections of short, standalone documents | News articles, customer support tickets, short contracts |
| **Semantic** | Splits text at natural meaning boundaries (topics, ideas). | Medium-High | Technical, academic, or narrative documents | Scientific papers, textbooks, novels, whitepapers |
| **LLM-Based** | Uses a language model to decide chunk boundaries based on context, meaning, or task needs. | High | Complex text where meaning-aware chunking improves downstream tasks like summarization or Q&A | Long reports, legal opinions, medical records |
| **Agentic** | Lets an AI agent decide how to split based on meaning and structure. | Very High | Complex, nuanced documents that require custom strategies | Regulatory filings, multi-section contracts, corporate policies |

*Source: Weaviate*

HANDS-ON MODULE 1

# Document Construction

→ Compare embedding the raw description alone vs. a constructed 'product card'.

→ Product card = title + brand + style code + subcategory + attributes + review snippets.

→ Measure recall specifically on the thin_description probe set — items with fluff descriptions.

→ Also write attributes into scalar fields — this sets up Module 3.

**DEMO QUERIES**

`"down jacket warm enough for zero degrees"`

**IDEAL** Surfaces the 0°F-rated down jackets — signal that lives only in attributes and reviews, not the fluff description.

`"stretchy work pants for men"`

**IDEAL** Surfaces the 4-way stretch chinos specifically, not just any pair of pants in the catalog.

**PROBE: thin_description**

*Re-run the scoreboard after this module — watch the recall and latency rows move.*

# 02

# Pre-Retrieval Optimization

Improving retrieval precision through smarter query understanding

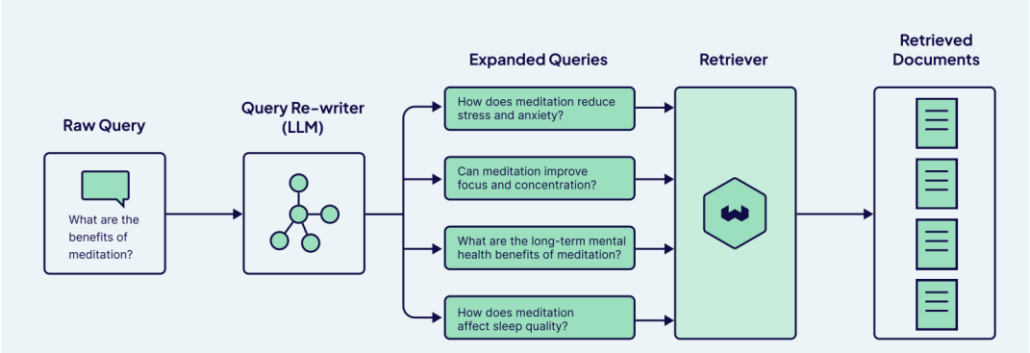
# Why Pre-Retrieval Optimization Matters

- Clarifies intent — reduces ambiguity in how the user actually phrased the request.
- Improves precision — the vector search receives a query aligned with how the corpus is written.
- Increases recall — catches relevant items that a literal query would miss entirely.
- Key idea: better queries lead to better retrieval, and better retrieval leads to better grounding.

# 2 Query Expansion

*Generate several phrasings of the same query and search with all of them.*

- Covers synonyms and common alternate phrasings a single rewrite might miss.
- Particularly effective against genuine vocabulary gaps between shopper and catalog language.
- Not free — expansion can hurt an already-precise query by diluting it with noise.




*Source: Weaviate*

1

# Query Rewriting

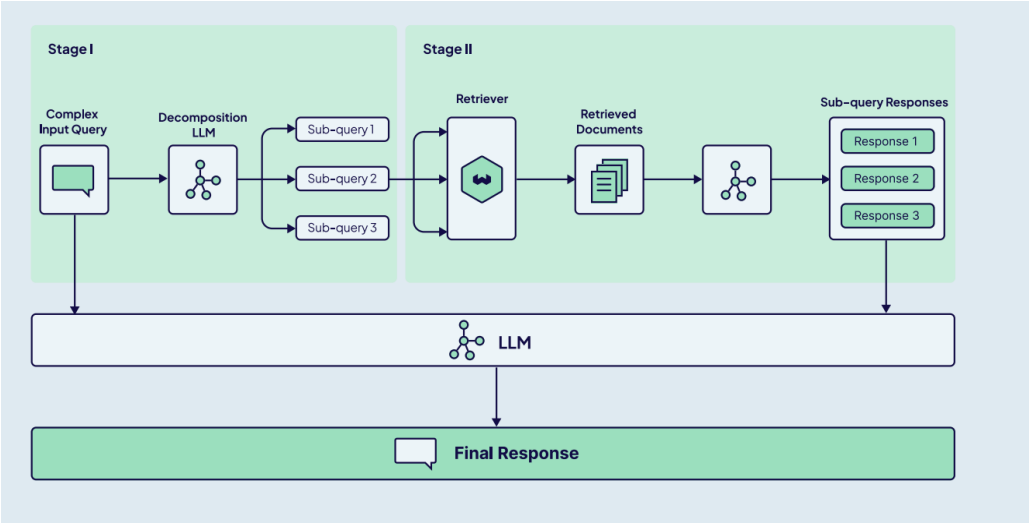
*Reformulate the query into the vocabulary the corpus was written in.*

- Shoppers say 'wedding outfit for a guy'; the catalog says 'Two-Piece Formal Suit'.
- An LLM rewrite bridges that vocabulary gap before the embedding step ever runs.
- Risk: an over-eager rewrite can drift from what the user actually meant.

# 3 Query Decomposition

*Split one query into independent sub-queries when it bundles multiple needs.*

- 'I need a suit and a dress shirt for my wedding' is two shopping needs, not one.
- Each sub-query retrieves independently; results are merged before generation.
- Without this, a bundled query tends to retrieve a blurred compromise that serves neither need well.




*Source: Weaviate*

HANDS-ON MODULE 2

# Query Decomposition

- → Implement expansion for the vocabulary gap between shopper phrasing and catalog language.
- → Implement decomposition for queries that bundle two distinct shopping needs.
- → Show the counter-example: expansion hurting a query that was already precise.
- → Query understanding is not free — an audience that only sees wins learns the wrong lesson.

**DEMO QUERY**

`"i need a suit and a dress shirt for my wedding"`

**IDEAL** Returns both a suit AND a separate dress shirt — two sub-results, not one blurred compromise.

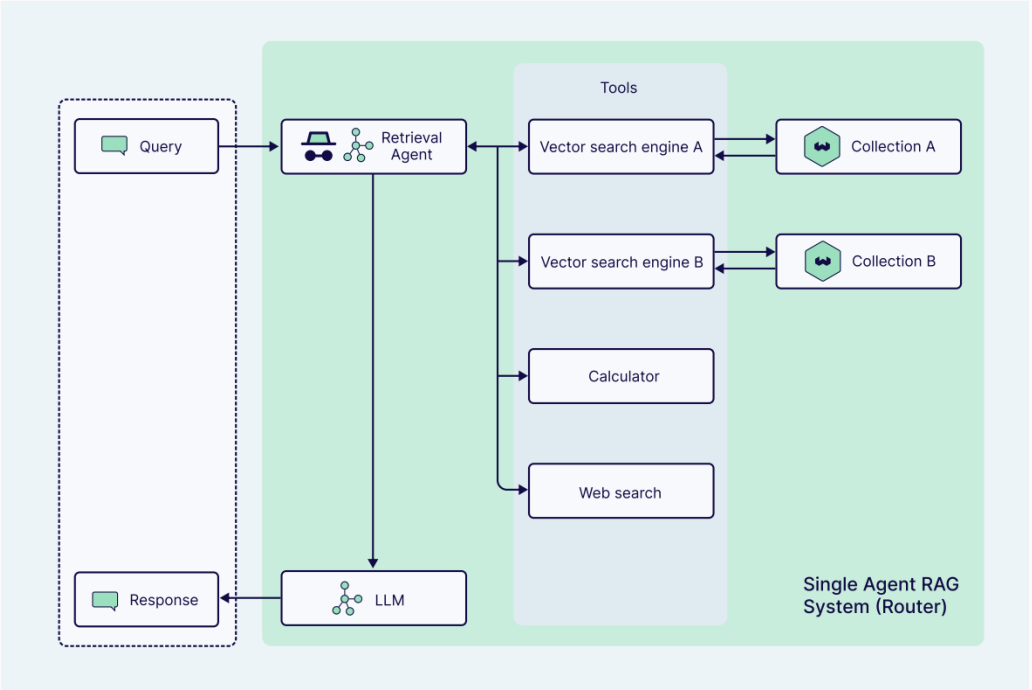
**PROBE: multi_intent (decomposition) · vocab_gap (expansion)**

*Re-run the scoreboard after this module — watch the recall and latency rows move.*

NOT BUILT TODAY

# Query Routing

Routing classifies a query's intent — product search, policy lookup, store hours — and sends it to the right retrieval path or corpus before anything else happens.




*Source: Weaviate*

**WHY NOT TODAY**

*Our corpus is product-only by design, so there is nothing else to route a query toward. Routing matters the moment you add a second corpus type — worth knowing the pattern exists.*

# 03

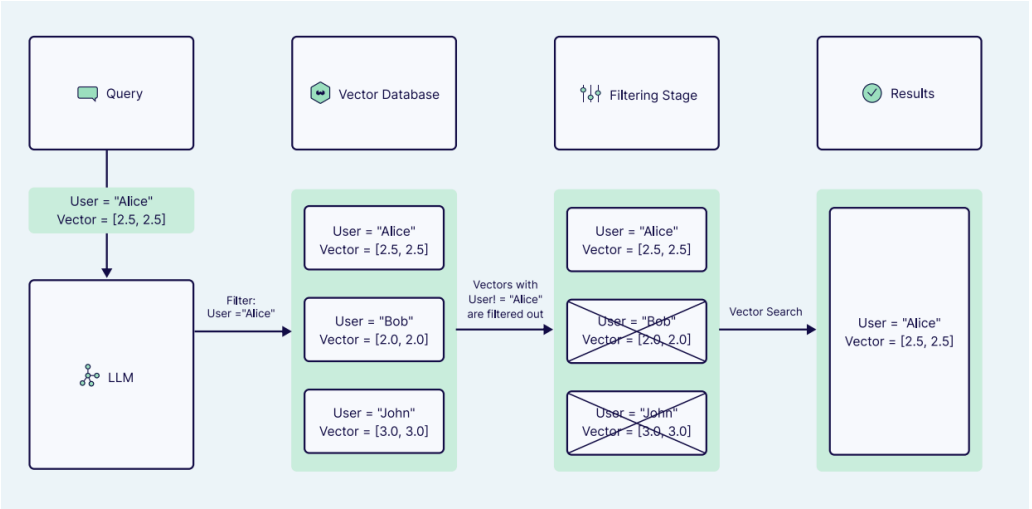
# Retrieval Optimization

Improving search relevance with optimized retrieval pipelines

# 1 Metadata Filtering

*Use structured attributes to narrow the candidate set before or alongside vector search.*

- Color, size and price aren't semantic properties — similarity search treats them as noise.
- A cluster of 56 near-identical running shoes is equally 'similar' to any query about running shoes.
- Filtering on extracted attributes is what actually answers 'red, size 10, under $50'.




*Source: Weaviate*

HANDS-ON MODULE 3

# Attribute Extraction to Filters

- → Extract {color, size, price_max, ...} from natural language into a Milvus filter expression.
- → Leave the semantic residue of the query for vector search on top of the filter.
- → Guard against an over-eager extractor returning zero rows — drop the weakest constraint and retry.
- → This is the single largest recall jump in the whole session.

DEMO QUERY

```
"red running shoes under $50 in size 10"
```

**IDEAL** The handful of SKUs that are actually red, size 10, and under $50 — not just 'running-shoe-flavored' results.

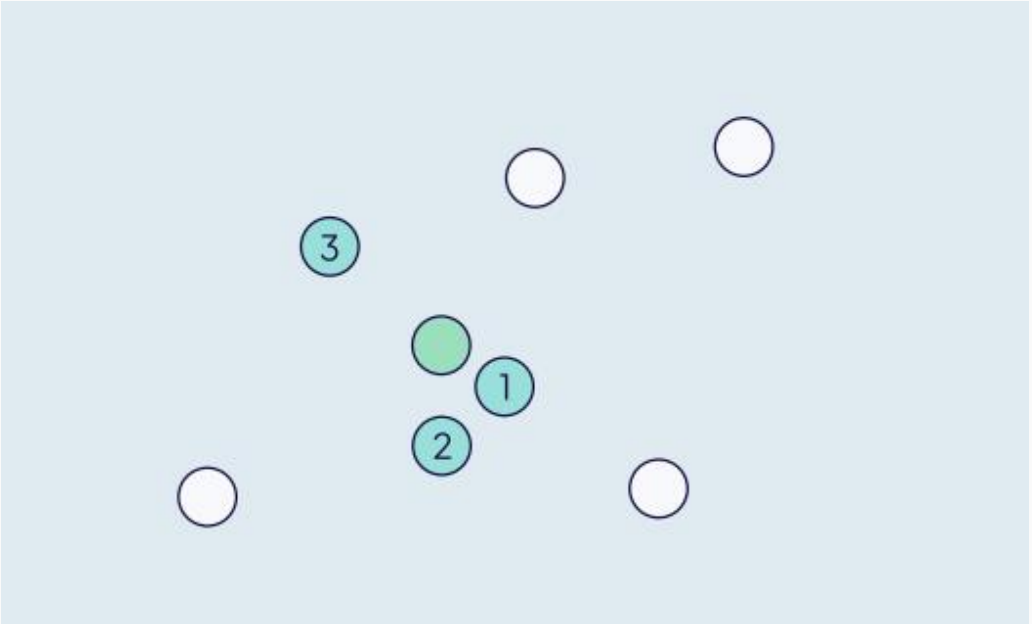
PROBE: attribute_constrained

*Re-run the scoreboard after this module — watch the recall and latency rows move.*

NOT BUILT TODAY

# Excluding Vector Search Outliers

Distance thresholding and autocut techniques drop low-similarity matches from the result set so obviously off-topic vectors never reach the LLM as context.




Source: Weaviate

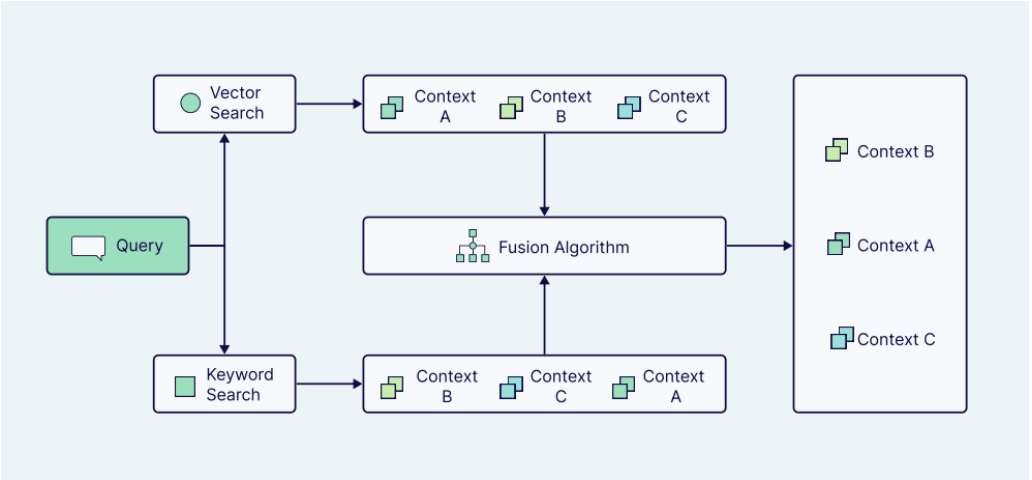
**WHY NOT TODAY**

*Our corpus is small and clean enough that off-topic noise isn't the dominant failure mode today. It matters more as corpus size and diversity grow.*

# 2 Hybrid Search

*Blend keyword-based (sparse) and semantic (dense) search to cover both exact and conceptual matches.*

- Dense embeddings capture meaning but blur distinctive tokens like model or style codes.
- Sparse search (BM25-style) nails exact tokens but misses paraphrase and synonymy entirely.
- Fusion — weighted or reciprocal rank — combines both into one ranked list.




*Source: Weaviate*

NOT BUILT TODAY

# Embedding Model Fine-Tuning

Retraining the embedding model on domain-specific text pairs so it represents your vocabulary natively — 'return period' sits near 'replacement window' instead of far from it.

WHY NOT TODAY

*It's the last lever, not the first fix. Most retrieval failures — including everything in this session — are cheaper to fix upstream of touching model weights.*

HANDS-ON MODULE 4

# Hybrid Search

- → Add sparse retrieval (BM25 or BGE-M3) alongside the existing dense index.
- → Compare WeightedRanker fusion against Reciprocal Rank Fusion.
- → Sweep the sparse/dense weight and plot recall — this is a tuned parameter, not a default.
- → Watch a near-miss style code lose to the exact match once sparse is in the mix.

**DEMO QUERY**

`"RN-4412"`

**IDEAL** The exact SKU with style code RN-4412, and only that one. Dense-only search returns near-miss codes RN-4142 / RN-4421 above it.

**PROBE: exact_token**

*Re-run the scoreboard after this module — watch the recall and latency rows move.*

# 04

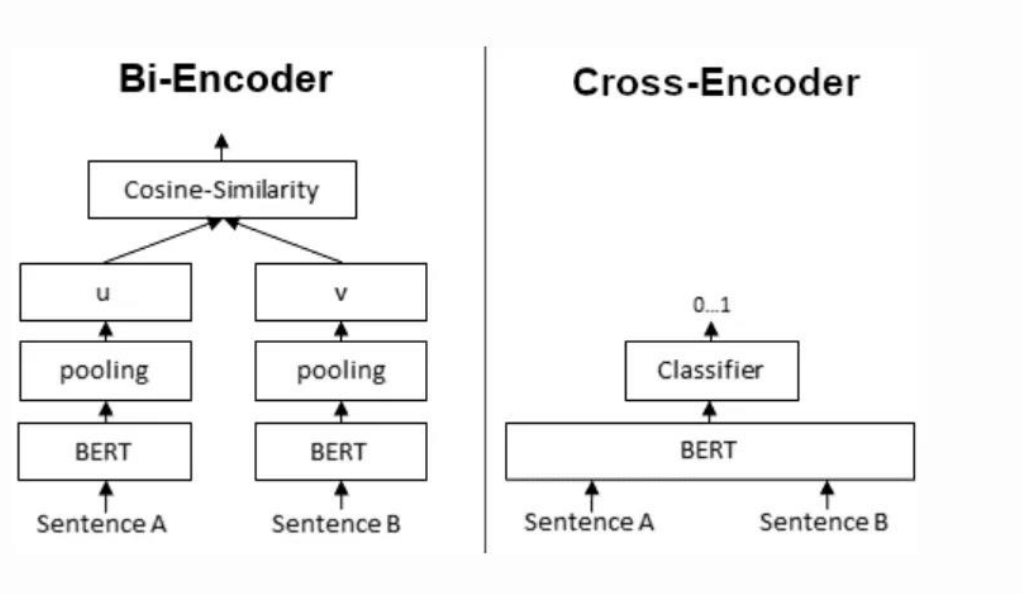
# Post-Retrieval Optimization

Turning good candidates into a good, grounded answer

# 1 Re-Ranking

*Retrieve a wide candidate set cheaply, then re-score it with a more expensive, more accurate model.*

- A bi-encoder embeds query and document separately — fast, but loses cross-interaction.
- A cross-encoder scores the query and document together — slower, but far more accurate at the top of the list.
- Two-stage retrieval: cheap recall at candidate_k, expensive precision at top_k.




*Source: Weaviate*

HANDS-ON MODULE 5

# Reranking with a Cross-Encoder

- → Retrieve 50 candidates, rerank down to the final 5 with ms-marco-MiniLM-L-6-v2.
- → Runs locally on CPU — no gateway call, no added per-query cost.
- → Watch the correct item travel from around rank 30 to rank 2.
- → Report the added latency against the recall gain — reranking isn't free even locally.

DEMO QUERY

```
"running shoes with a lot of cushioning
that are still lightweight"
```

**IDEAL** The one SKU with max cushioning under 9oz jumps from around rank 30 to rank 1–2 after reranking.

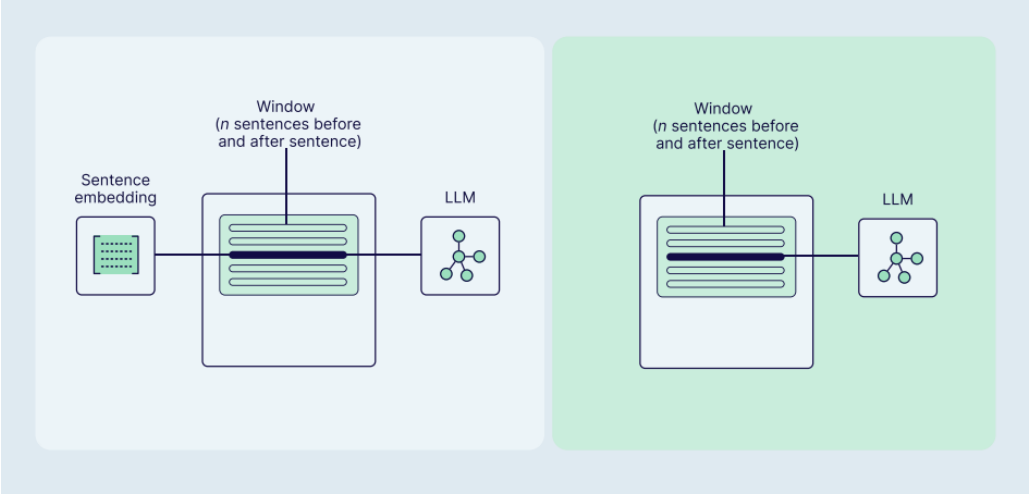
PROBE: `deep_rank`

*Re-run the scoreboard after this module — watch the recall and latency rows move.*

# 2 Context Post-Processing & Prompt Engineering

*Shape what actually reaches the prompt — and how the prompt uses it.*

- Dedup near-identical items — eight colourways of one shirt shouldn't fill all five slots.
- Diversity — enforce that the final set spans genuinely different options.
- Grounded prompting — require citations and a refusal when nothing clears a relevance floor.
- An ungrounded prompt will confidently recommend a product that isn't in the catalog. That's the failure that ends a retail assistant project.




*Source: Weaviate*

HANDS-ON MODULE 6

# Context Assembly & Grounded Generation

- → Dedup near-duplicate colourway clusters down to one entry with a variant note.
- → Apply a diversity penalty so the final top-5 spans real options, not one item eight ways.
- → Require sku_id citations in generation, with an explicit refusal path.
- → Show the hallucinated recommendation first, then the refusal behavior fixing it.

**DEMO QUERIES**

`"plain cotton t shirt for men"`

**IDEAL** One representative tee, not all eight colourways of the same shirt filling every slot.

`"crew socks"`

**IDEAL** A genuinely diverse top-5, not one sock pack repeated five times over.

**PROBE: near_duplicate**

*Re-run the scoreboard after this module — watch the recall and latency rows move.*

NOT BUILT TODAY

# LLM Fine-Tuning

Retraining the generation model's own weights on your data — new tone, new domain vocabulary, new tasks it can perform outright, not just better context.

**WHY NOT TODAY**

*Prompting changes how you ask. RAG changes what the model knows at query time. Fine-tuning changes what it can do at all — a different lever from everything built today.*

# Thank You

If you have any questions, feel free to ping on Teams

Facilitated by: Tanmay Dhote